Academia de Studii Economice, București
Facultatea de Cibernetică, Statistică și Informatică Economică
Specializarea: Informatică Economică

Proiect SGBD ORACLE

Tema proiectului: Gestiune Sezoane Formula 1

Student: Pirvan Sebastian-Daniel
Grupa: 1057

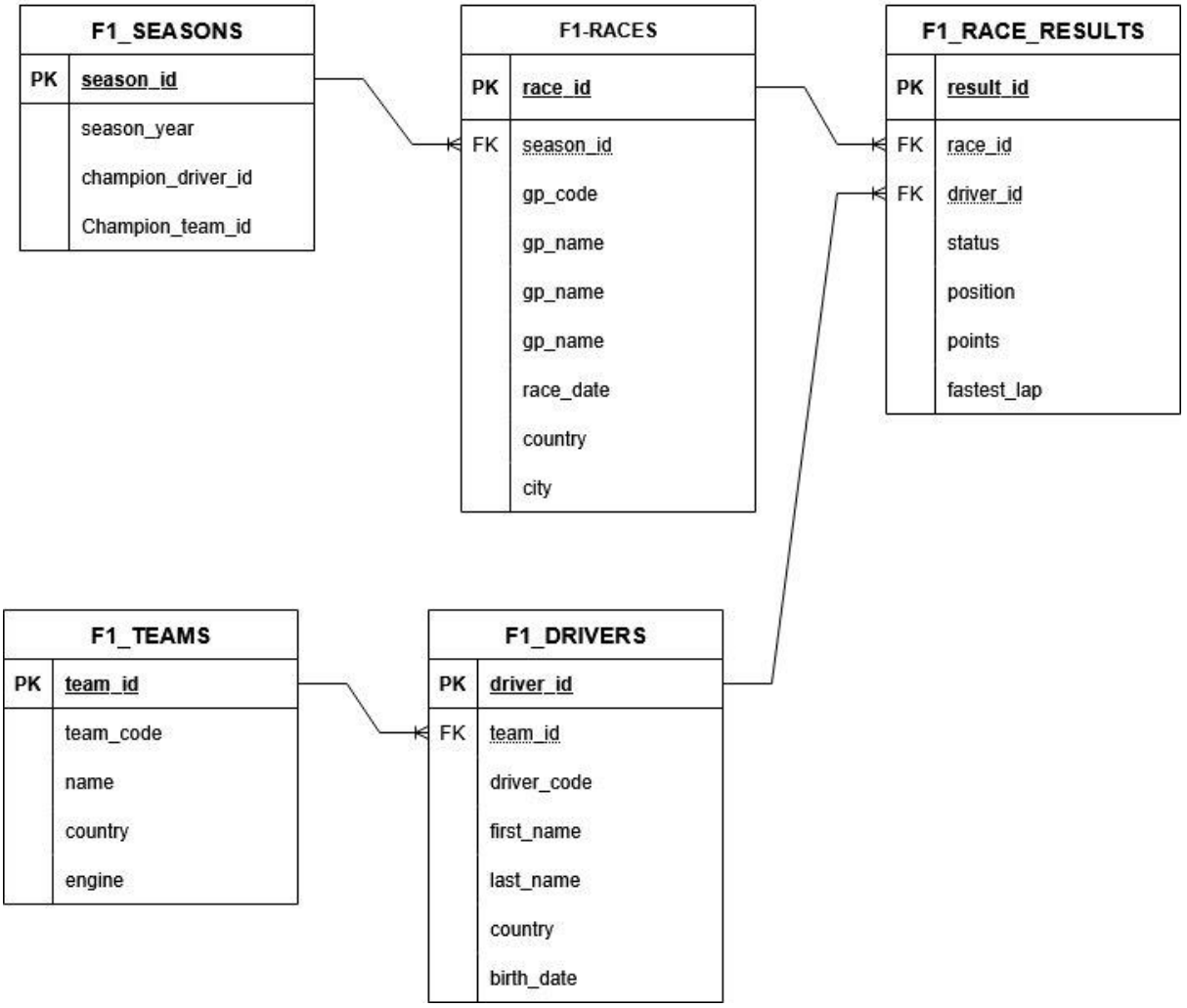
Descrierea problemei și prezentarea schemei conceptuale a tabelelor

Să se modeleze printr-o bază de date un sezon de Formula 1, astfel încât să se poată gestiona sezoane, curse, echipe, piloți și rezultatele din curse. Modelul trebuie să permită calculul punctajului conform regulilor din F1 și stabilirea campionilor piloti si echipe ai sezonului.

Schema conceptuală include tabelele: **F1_SEASONS, F1_RACES, F1_TEAMS, F1_DRIVERS, F1_RACE_RESULTS**

Relații:

- un sezon are mai multe curse (1:N),
- o cursă are mai multe rezultate (1:N),
- o echipă are mai mulți piloți (1:N),
- un pilot are rezultate în mai multe curse (1:N).



Interacțiunea cu serverul Oracle prin comenzile SQL

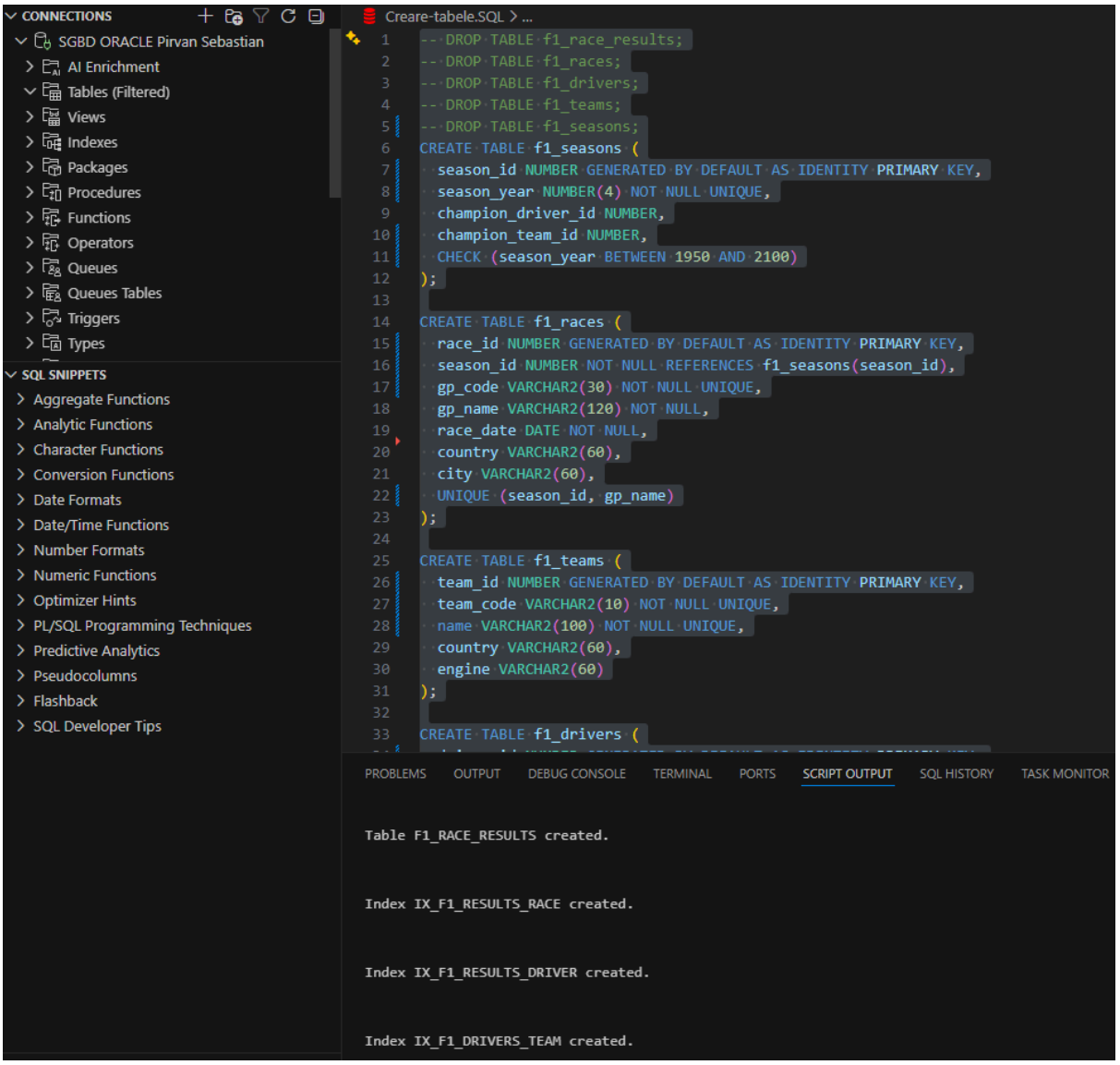
Să se implementeze în Oracle schema proiectată, folosind comenzi LDD pentru crearea tabelelor și constrângerilor și comenzi LMD pentru popularea bazei de date cu date de test. Pentru verificare se vor rula interogări SELECT.

Creare Tabele:

```
CREATE TABLE f1_seasons (
  season_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  season_year NUMBER(4) NOT NULL UNIQUE,
  champion_driver_id NUMBER,
  champion_team_id NUMBER,
  CHECK (season_year BETWEEN 1950 AND 2100)
);


CREATE TABLE f1_races (
  race_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  season_id NUMBER NOT NULL REFERENCES f1_seasons(season_id),
  gp_code VARCHAR2(30) NOT NULL UNIQUE,
  gp_name VARCHAR2(120) NOT NULL,
  race_date DATE NOT NULL,
  country VARCHAR2(60),
  city VARCHAR2(60),
  UNIQUE (season_id, gp_name)
);


CREATE TABLE f1_teams (
```

```
  team_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  team_code VARCHAR2(10) NOT NULL UNIQUE,
  name VARCHAR2(100) NOT NULL UNIQUE,
  country VARCHAR2(60),
  engine VARCHAR2(60)
);

CREATE TABLE f1_drivers (
  driver_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  driver_code VARCHAR2(10) NOT NULL UNIQUE,
  team_id NUMBER NOT NULL REFERENCES f1_teams(team_id),
  first_name VARCHAR2(60) NOT NULL,
  last_name VARCHAR2(60) NOT NULL,
  country VARCHAR2(60),
  birth_date DATE
);

CREATE TABLE f1_race_results (
  result_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  race_id NUMBER NOT NULL REFERENCES f1_races(race_id),
  driver_id NUMBER NOT NULL REFERENCES f1_drivers(driver_id),
  status VARCHAR2(10) NOT NULL,
  position NUMBER(2),
  points NUMBER(3) DEFAULT 0 NOT NULL,
  fastest_lap CHAR(1) DEFAULT 'N' NOT NULL,
```

```
  CHECK (status IN ('FINISH','DNF','DSQ')),

  CHECK (fastest_lap IN ('Y','N')),

  CHECK (points >= 0),

  UNIQUE (race_id, driver_id)

);
```

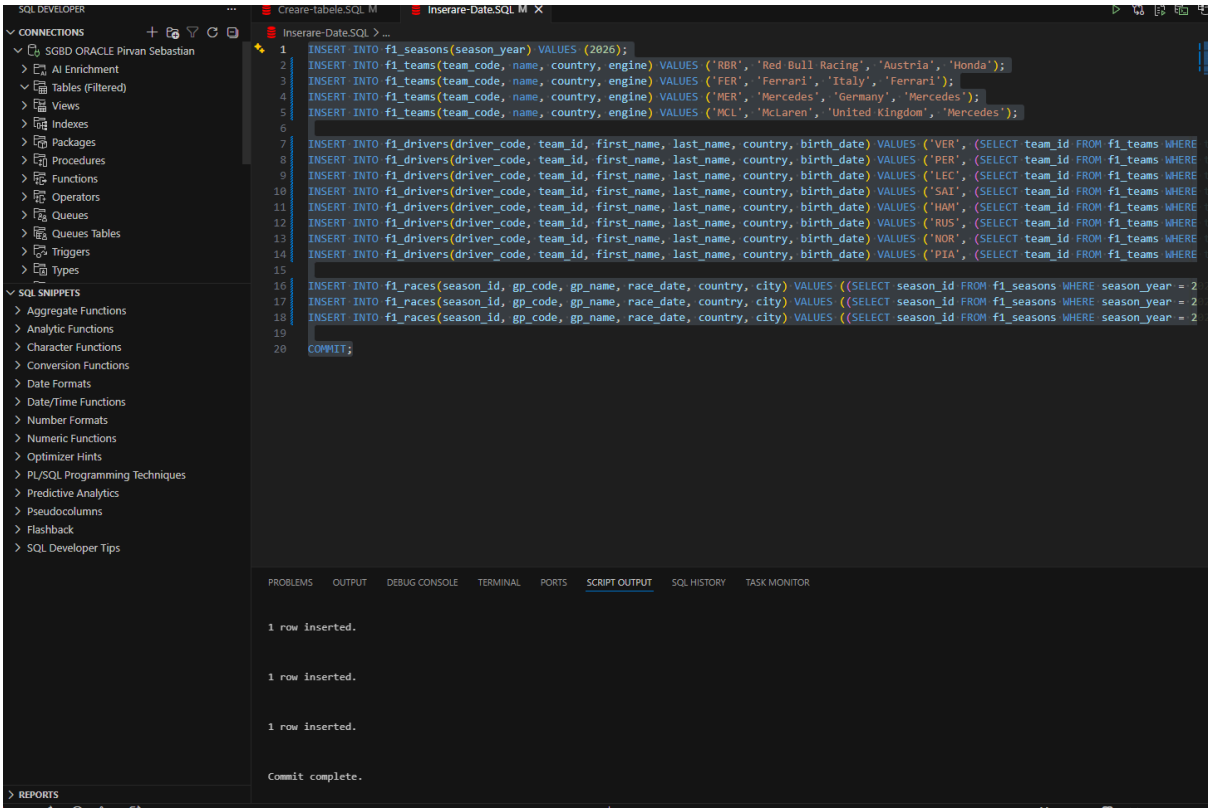


Inserare Date:

```
INSERT INTO f1_seasons(season_year) VALUES (2026);
INSERT INTO f1_teams(team_code, name, country, engine) VALUES ('RBR',
'Red Bull Racing', 'Austria', 'Honda');
```

```sql
INSERT INTO f1_teams(team_code, name, country, engine) VALUES ('FER',
'Ferrari', 'Italy', 'Ferrari');
INSERT INTO f1_teams(team_code, name, country, engine) VALUES ('MER',
'Mercedes', 'Germany', 'Mercedes');
INSERT INTO f1_teams(team_code, name, country, engine) VALUES ('MCL',
'McLaren', 'United Kingdom', 'Mercedes');

INSERT INTO f1_drivers(driver_code, team_id, first_name, last_name,
country, birth_date) VALUES ('VER', (SELECT team_id FROM f1_teams WHERE
team_code='RBR'), 'Max', 'Verstappen', 'Netherlands', DATE
'1997-09-30');
INSERT INTO f1_drivers(driver_code, team_id, first_name, last_name,
country, birth_date) VALUES ('PER', (SELECT team_id FROM f1_teams WHERE
team_code='RBR'), 'Sergio', 'Perez', 'Mexico', DATE '1990-01-26');
INSERT INTO f1_drivers(driver_code, team_id, first_name, last_name,
country, birth_date) VALUES ('LEC', (SELECT team_id FROM f1_teams WHERE
team_code='FER'), 'Charles', 'Leclerc', 'Monaco', DATE '1997-10-16');
INSERT INTO f1_drivers(driver_code, team_id, first_name, last_name,
country, birth_date) VALUES ('SAI', (SELECT team_id FROM f1_teams WHERE
team_code='FER'), 'Carlos', 'Sainz', 'Spain', DATE '1994-09-01');
INSERT INTO f1_drivers(driver_code, team_id, first_name, last_name,
country, birth_date) VALUES ('HAM', (SELECT team_id FROM f1_teams WHERE
team_code='MER'), 'Lewis', 'Hamilton', 'United Kingdom', DATE
'1985-01-07');
INSERT INTO f1_drivers(driver_code, team_id, first_name, last_name,
country, birth_date) VALUES ('RUS', (SELECT team_id FROM f1_teams WHERE
team_code='MER'), 'George', 'Russell', 'United Kingdom', DATE
'1998-02-15');
INSERT INTO f1_drivers(driver_code, team_id, first_name, last_name,
country, birth_date) VALUES ('NOR', (SELECT team_id FROM f1_teams WHERE
team_code='MCL'), 'Lando', 'Norris', 'United Kingdom', DATE
'1999-11-13');
INSERT INTO f1_drivers(driver_code, team_id, first_name, last_name,
country, birth_date) VALUES ('PIA', (SELECT team_id FROM f1_teams WHERE
team_code='MCL'), 'Oscar', 'Piastri', 'Australia', DATE '2001-04-06');

INSERT INTO f1_races(season_id, gp_code, gp_name, race_date, country,
city) VALUES ((SELECT season_id FROM f1_seasons WHERE season_year =
2026), 'BAHRAIN_2026', 'Bahrain GP', DATE '2026-03-08', 'Bahrain',
'Sakhir');
INSERT INTO f1_races(season_id, gp_code, gp_name, race_date, country,
city) VALUES ((SELECT season_id FROM f1_seasons WHERE season_year =
```

```sql
2026), 'EMILIA_2026', 'Emilia Romagna GP', DATE '2026-05-17', 'Italy',
'Imola');
INSERT INTO f1_races(season_id, gp_code, gp_name, race_date, country,
city) VALUES ((SELECT season_id FROM f1_seasons WHERE season_year =
2026), 'MONACO_2026', 'Monaco GP', DATE '2026-05-24', 'Monaco', 'Monte
Carlo');

COMMIT;
```



Testare Date:

```
SELECT driver_code, first_name, last_name FROM f1_drivers;

SHOW USER;
SELECT USER FROM dual;

USER is "PIRVANM_57"
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT

All rows fetched: 8 in 0.031 seconds

	DRIVER_CODE	FIRST_NAME	LAST_NAME
1	VER	Max	Verstappen
2	PER	Sergio	Perez
3	LEC	Charles	Leclerc
4	SAI	Carlos	Sainz
5	HAM	Lewis	Hamilton
6	RUS	George	Russell
7	NOR	Lando	Norris
8	PIA	Oscar	Piastri

Structuri alternative și repetitive

Să se calculeze punctajul pentru un rezultat de cursă în funcție de poziția obținută. Să se valideze rezultatele introduse astfel încât un pilot cu **FINISH** să aibă obligatoriu o poziție, iar un pilot cu **DNF/DSQ** să nu aibă poziție. Să se recalculeze punctele pentru toți piloții dintr-o cursă prin parcurgerea repetitivă a rezultatelor și actualizarea punctelor în tabel.

```
FOR rec IN c_res(v_race_id) LOOP

      IF rec.status = 'FINISH' THEN

        v_base_points := fn_points_for_position(rec.position);

        IF rec.fastest_lap = 'Y' THEN

          IF rec.position BETWEEN 1 AND 10 THEN v_base_points :=
v_base_points + 1; ELSE RAISE e_fastest_lap_rule; END IF;

        END IF;
```

```
        UPDATE f1_race_results SET points = v_base_points WHERE CURRENT
OF c_res;

      ELSE

        UPDATE f1_race_results SET points = 0 WHERE CURRENT OF c_res;

      END IF;

    END LOOP;
```

Creare-tabele.SQL | Triggers.sql | Inserare-Date.SQL | testare.sql

```
BEGIN
pkg_f1.pr_recalc_race_points('MONACO_2026');
  DBMS_OUTPUT.PUT_LINE('Puncte recalcualte pentru Monaco');
END;
/
-- USER is "PIRVANM_57";
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | QUERY RESULT | SCRIPT OUT

```
Puncte recalcualte pentru Monaco


PL/SQL procedure successfully completed.
```

```
CASE p_position

      WHEN 1  THEN 25

      WHEN 2  THEN 18

      WHEN 3  THEN 15

      WHEN 4  THEN 12

      WHEN 5  THEN 10

      WHEN 6  THEN 8

      WHEN 7  THEN 6

      WHEN 8  THEN 4
```

```
      WHEN 9  THEN 2

      WHEN 10 THEN 1

      ELSE 0

    END;
```

```
SELECT
pkg_f1.fn_points_for_position(1)  AS pozitie1,
  pkg_f1.fn_points_for_position(2)  AS pozitie2,
  pkg_f1.fn_points_for_position(10) AS pozitie10,
  pkg_f1.fn_points_for_position(11) AS pozitie11
FROM dual;

-- USER is "PIRVANM_57";
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT

All rows fetched: 1 in 0.047 seconds

	POZITIE1	POZITIE2	POZITIE10	POZITIE11
1	25	18	1	0

Tratarea excepțiilor

Să se trateze excepții implicite și excepții explicite definite de utilizator, pentru a gestiona situațiile invalide. Să se trateze cand nu se găsește șoferul/cursa căutată și inserarea dubla pe Unique

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20001, 'Race gp_code or driver_code not
found.');
    WHEN DUP_VAL_ON_INDEX THEN
      RAISE_APPLICATION_ERROR(-20002, 'Duplicate result: this driver
already has a result for this race.');
    WHEN e_invalid_position THEN
      RAISE_APPLICATION_ERROR(-20010, 'Invalid position for status.');
    WHEN e_fastest_lap_rule THEN
      RAISE_APPLICATION_ERROR(-20011, 'Invalid fastest_lap value (must
be Y or N).'); --YES or NO
  END pr_add_result;
```

```
BEGIN
pkg_f1.pr_add_result('TEST','VER','FINISH',1,'N');
END;
/
-- USER is "PIRVANM_57";
```

EMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESU

```
N

R at line 1:
20001: Race gp_code or driver_code not found.
```

```
BEGIN
pkg_f1.pr_add_result('MONACO_2026','VER','FINISH',1,'N');
pkg_f1.pr_add_result('MONACO_2026','VER','FINISH',1,'N');
END;
/
-- USER is "PIRVANM_57";
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPU

```
BEGIN
*
ERROR at line 1:
ORA-20002: Duplicate result: this driver already has a result for this race.
ORA-06512: at "PIRVANM_57.PKG_F1", line 79
```

```
BEGIN
  pkg_f1.pr_add_result('MONACO_2026','PER','DNF',5,'N');
END;
/
-- USER is "PIRVANM_57";
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SC

```
BEGIN
*
ERROR at line 1:
ORA-20010: Invalid position for status.
ORA-06512: at "PIRVANM_57.PKG_F1", line 81
ORA-06512: at line 2
```

```
BEGIN
  pkg_f1.pr_add_result('MONACO_2026','LEC','FINISH',2,'X');
END;
/
-- USER is "PIRVANM_57";
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT C

```
BEGIN
*
ERROR at line 1:
ORA-20011: Invalid fastest_lap value (must be Y or N).
ORA-06512: at "PIRVANM_57.PKG_F1", line 83
ORA-06512: at line 2
```

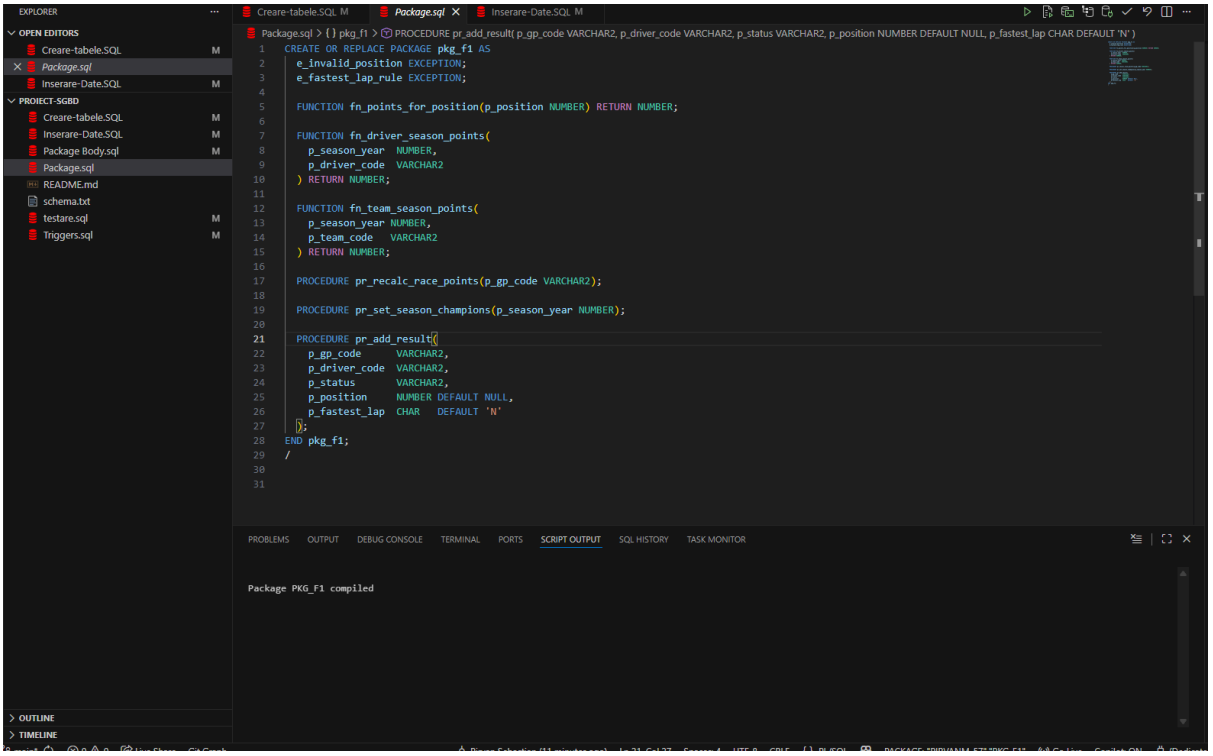
Funcții, proceduri, includerea acestora în pachete

Să se centralizeze într-un pachet logica, astfel încât punctajul să poată fi calculat uniform pentru orice poziție, totalurile de puncte pe sezon să poată fi obținute pentru un pilot sau o echipă, punctele unei curse să poată fi recalculate automat pentru toți participanții și campionii sezonului să fie determinați pe baza punctelor acumulate

Package:

```
CREATE OR REPLACE PACKAGE pkg_f1 AS
```

```
  e_invalid_position EXCEPTION;

  e_fastest_lap_rule EXCEPTION;

  FUNCTION fn_points_for_position(p_position NUMBER) RETURN NUMBER;

  FUNCTION fn_driver_season_points(

    p_season_year  NUMBER,

    p_driver_code  VARCHAR2

  ) RETURN NUMBER;

  FUNCTION fn_team_season_points(

    p_season_year NUMBER,

    p_team_code   VARCHAR2

  ) RETURN NUMBER;



  PROCEDURE pr_recalc_race_points(p_gp_code VARCHAR2);

  PROCEDURE pr_set_season_champions(p_season_year NUMBER);

  PROCEDURE pr_add_result(

    p_gp_code      VARCHAR2,

    p_driver_code  VARCHAR2,

    p_status       VARCHAR2,

    p_position     NUMBER DEFAULT NULL,

    p_fastest_lap  CHAR   DEFAULT 'N'

  );

END pkg_f1;

/
```



Package Body:

```oracle_plsql
CREATE OR REPLACE PACKAGE BODY pkg_f1 AS
  FUNCTION fn_points_for_position(p_position NUMBER) RETURN NUMBER IS
  BEGIN
    RETURN CASE p_position
      WHEN 1  THEN 25
      WHEN 2  THEN 18
      WHEN 3  THEN 15
      WHEN 4  THEN 12
      WHEN 5  THEN 10
      WHEN 6  THEN 8
      WHEN 7  THEN 6
      WHEN 8  THEN 4
      WHEN 9  THEN 2
      WHEN 10 THEN 1
      ELSE 0
    END;
  END fn_points_for_position;


  FUNCTION fn_driver_season_points(
    p_season_year  NUMBER,
    p_driver_code  VARCHAR2
  ) RETURN NUMBER IS
```

```
    v_points NUMBER;
  BEGIN
    SELECT NVL(SUM(rr.points), 0)
      INTO v_points
      FROM f1_race_results rr
      JOIN f1_races r   ON r.race_id = rr.race_id
      JOIN f1_seasons s ON s.season_id = r.season_id
      JOIN f1_drivers d ON d.driver_id = rr.driver_id
     WHERE s.season_year = p_season_year
       AND d.driver_code = p_driver_code;

    RETURN v_points;
  END fn_driver_season_points;
  FUNCTION fn_team_season_points(p_season_year NUMBER, p_team_code
VARCHAR2) RETURN NUMBER IS
    v_points NUMBER;
  BEGIN
    SELECT NVL(SUM(rr.points), 0)
      INTO v_points
      FROM f1_race_results rr
      JOIN f1_races r   ON r.race_id = rr.race_id
      JOIN f1_seasons s ON s.season_id = r.season_id
      JOIN f1_drivers d ON d.driver_id = rr.driver_id
      JOIN f1_teams t   ON t.team_id = d.team_id
     WHERE s.season_year = p_season_year
       AND t.team_code   = p_team_code;

    RETURN v_points;
  END fn_team_season_points;
  PROCEDURE pr_add_result(p_gp_code VARCHAR2, p_driver_code VARCHAR2,
p_status VARCHAR2, p_position NUMBER DEFAULT NULL, p_fastest_lap CHAR
DEFAULT 'N') IS
    v_race_id f1_races.race_id%TYPE;
    v_driver_id f1_drivers.driver_id%TYPE;
  BEGIN
    SELECT race_id INTO v_race_id FROM f1_races WHERE gp_code =
p_gp_code;
    SELECT driver_id INTO v_driver_id FROM f1_drivers WHERE driver_code
= p_driver_code;

    IF p_status = 'FINISH' THEN
      IF p_position IS NULL OR p_position < 1 THEN
        RAISE e_invalid_position;
```

```
      END IF;
    ELSE
      IF p_position IS NOT NULL THEN
        RAISE e_invalid_position;
      END IF;
    END IF;

    IF p_fastest_lap NOT IN ('Y','N') THEN
      RAISE e_fastest_lap_rule;
    END IF;
    INSERT INTO f1_race_results(race_id, driver_id, status, position,
points, fastest_lap)
    VALUES (v_race_id, v_driver_id, p_status, p_position, 0,
NVL(p_fastest_lap,'N'));

  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20001, 'Race gp_code or driver_code not
found.');
    WHEN DUP_VAL_ON_INDEX THEN
      RAISE_APPLICATION_ERROR(-20002, 'Duplicate result: this driver
already has a result for this race.');
    WHEN e_invalid_position THEN
      RAISE_APPLICATION_ERROR(-20010, 'Invalid position for given
status (FINISH needs position; DNF/DSQ must have NULL).');
    WHEN e_fastest_lap_rule THEN
      RAISE_APPLICATION_ERROR(-20011, 'Invalid fastest_lap value (must
be Y or N).');
  END pr_add_result;
  PROCEDURE pr_recalc_race_points(p_gp_code VARCHAR2) IS
    v_race_id f1_races.race_id%TYPE;

    CURSOR c_res(p_race_id NUMBER) IS
      SELECT result_id, status, position, fastest_lap
        FROM f1_race_results
       WHERE race_id = p_race_id
       FOR UPDATE OF points;

    v_base_points NUMBER;
  BEGIN
    SELECT race_id INTO v_race_id FROM f1_races WHERE gp_code =
p_gp_code;
```

```
    FOR rec IN c_res(v_race_id) LOOP
      IF rec.status = 'FINISH' THEN
        v_base_points := fn_points_for_position(rec.position);
        IF rec.fastest_lap = 'Y' THEN
          IF rec.position BETWEEN 1 AND 10 THEN v_base_points :=
v_base_points + 1; ELSE RAISE e_fastest_lap_rule; END IF;
        END IF;
        UPDATE f1_race_results SET points = v_base_points WHERE CURRENT
OF c_res;
      ELSE
        UPDATE f1_race_results SET points = 0 WHERE CURRENT OF c_res;
      END IF;
    END LOOP;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20003, 'Race not found for gp_code=' ||
p_gp_code);
    WHEN e_fastest_lap_rule THEN
      RAISE_APPLICATION_ERROR(-20012, 'Fastest lap is only allowed for
a FINISH result in top 10.');
  END pr_recalc_race_points;
  PROCEDURE pr_set_season_champions(p_season_year NUMBER) IS
    v_driver_id NUMBER;
    v_team_id NUMBER;
  BEGIN
    SELECT driver_id
      INTO v_driver_id
      FROM (
        SELECT d.driver_id, SUM(rr.points) pts
          FROM f1_drivers d
          JOIN f1_race_results rr ON rr.driver_id = d.driver_id
          JOIN f1_races r         ON r.race_id = rr.race_id
          JOIN f1_seasons s       ON s.season_id = r.season_id
         WHERE s.season_year = p_season_year
         GROUP BY d.driver_id
         ORDER BY pts DESC, d.driver_id ASC
      )
     WHERE ROWNUM = 1;
    SELECT team_id
      INTO v_team_id
      FROM (
        SELECT t.team_id, SUM(rr.points) pts
          FROM f1_teams t
```

```
          JOIN f1_drivers d      ON d.team_id = t.team_id
          JOIN f1_race_results rr ON rr.driver_id = d.driver_id
          JOIN f1_races r         ON r.race_id = rr.race_id
          JOIN f1_seasons s       ON s.season_id = r.season_id
         WHERE s.season_year = p_season_year
         GROUP BY t.team_id
         ORDER BY pts DESC, t.team_id ASC
      )
     WHERE ROWNUM = 1;

    UPDATE f1_seasons
       SET champion_driver_id = v_driver_id,
           champion_team_id   = v_team_id
     WHERE season_year = p_season_year;

    IF SQL%ROWCOUNT = 0 THEN
      RAISE_APPLICATION_ERROR(-20004, 'Season not found: ' ||
p_season_year);
    END IF;

  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20005, 'No race results found for season
' || p_season_year);
  END pr_set_season_champions;

END pkg_f1;
/
SHOW ERRORS
```

```sql
CREATE OR REPLACE PACKAGE BODY pkg_f1 AS
FUNCTION fn_points_for_position(p_position NUMBER) RETURN NUMBER IS
BEGIN
    RETURN CASE p_position
        WHEN 1  THEN 25
        WHEN 2  THEN 18
        WHEN 3  THEN 15
        WHEN 4  THEN 12
        WHEN 5  THEN 10
        WHEN 6  THEN 8
        WHEN 7  THEN 6
        WHEN 8  THEN 4
        WHEN 9  THEN 2
        WHEN 10 THEN 1
        ELSE 0
    END;
END fn_points_for_position;


FUNCTION fn_driver_season_points(
    p_season_year  NUMBER,
    p_driver_code  VARCHAR2
) RETURN NUMBER IS
    v_points NUMBER;
BEGIN
    SELECT NVL(SUM(rr.points), 0)
        INTO v_points
        FROM f1_race_results rr
        JOIN f1_races r  ON r.race_id = rr.race_id
        JOIN f1_seasons s ON s.season_id = r.season_id
        JOIN f1_drivers d ON d.driver_id = rr.driver_id
    WHERE s.season_year = p_season_year
        AND d.driver_code = p_driver_code;
```

```
Package Body PKG_F1 compiled

No errors.
```

```sql
SELECT pkg_f1.fn_points_for_position(1) AS p1 FROM dual;
USER is "PIRVANM_57"
```

All rows fetched: 1 in 0.027 seconds

	P1
1	25

Declanșatori

Să se prevină la nivelul bazei de date introducerea de rezultate care ar încălca reguli esențiale ale aplicației

- fastest_lap poate avea doar valorile Y/N și este acordat doar unui FINISH.
- statusul rezultatului trebuie să fie consistent cu poziția și punctele trebuie să rămână 0 pentru DNF/DSQ.

Triggers.sql > ...

```
CREATE OR REPLACE TRIGGER trg_f1_validate_fastest_lap
BEFORE INSERT OR UPDATE OF fastest_lap, status ON f1_race_results
FOR EACH ROW
BEGIN
  IF :NEW.fastest_lap IS NULL THEN
    :NEW.fastest_lap := 'N';
  END IF;
  IF :NEW.fastest_lap NOT IN ('Y', 'N') THEN
    RAISE_APPLICATION_ERROR(-20120, 'fastest_lap must be Y or N.');
  END IF;
  IF :NEW.status <> 'FINISH' THEN
    :NEW.fastest_lap := 'N';
  END IF;
END;
/
CREATE OR REPLACE TRIGGER trg_f1_validate_status_position BEFORE INSERT OR UPDATE OF status, position, points ON f1_race_results FOR EACH ROW
BEGIN
  IF :NEW.status = 'FINISH' THEN
    IF :NEW.position IS NULL OR :NEW.position < 1 THEN
      RAISE_APPLICATION_ERROR(-20110, 'FINISH requires a valid position (>=1).');
    END IF;
  ELSE
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR

```
Trigger TRG_F1_VALIDATE_FASTEST_LAP compiled

Trigger TRG_F1_VALIDATE_STATUS_POSITION compiled

Pirvan Sebastian

PL/SQL procedure successfully completed.
```

TRG_F1_VALIDATE_FASTEST_LAP:

```
CREATE OR REPLACE TRIGGER trg_f1_validate_fastest_lap

BEFORE INSERT OR UPDATE OF fastest_lap, status ON f1_race_results

FOR EACH ROW

BEGIN

  IF :NEW.fastest_lap IS NULL THEN

    :NEW.fastest_lap := 'N';

  END IF;

  IF :NEW.fastest_lap NOT IN ('Y', 'N') THEN

    RAISE_APPLICATION_ERROR(-20120, 'fastest_lap must be Y or N.');

  END IF;

  IF :NEW.status <> 'FINISH' THEN

    :NEW.fastest_lap := 'N';

  END IF;

END;
```

Creare-tabele.SQL | Triggers.sql M | Inserare-Date.SQL | testare.sql | UPDATE f1_race_results Untitled-1

```sql
UPDATE f1_race_results
   SET fastest_lap = 'X'
 WHERE race_id = (SELECT race_id FROM f1_races WHERE gp_code='MONACO_2026')
   AND driver_id = (SELECT driver_id FROM f1_drivers WHERE driver_code='LEC');
-- USER is "PIRVANM_57";
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | QUERY RESULT | SCRIPT OUTPUT | SQL HISTORY | TASK MONITOR

```
Error starting at line : 1 in command -
UPDATE f1_race_results
   SET fastest_lap = 'X'
 WHERE race_id = (SELECT race_id FROM f1_races WHERE gp_code='MONACO_2026')
   AND driver_id = (SELECT driver_id FROM f1_drivers WHERE driver_code='LEC')
Error at Command Line : 1 Column : 8
Error report -
SQL Error: ORA-20120: fastest_lap must be Y or N.
ORA-06512: at "PIRVANM_57.TRG_F1_VALIDATE_FASTEST_LAP", line 6
ORA-04088: error during execution of trigger 'PIRVANM_57.TRG_F1_VALIDATE_FASTEST_LAP'
```

Trg_f1_validate_status_position:

```sql
CREATE OR REPLACE TRIGGER trg_f1_validate_status_position BEFORE INSERT
OR UPDATE OF status, position, points ON f1_race_results FOR EACH ROW
BEGIN
  IF :NEW.status = 'FINISH' THEN
    IF :NEW.position IS NULL OR :NEW.position < 1 THEN
      RAISE_APPLICATION_ERROR(-20110, 'FINISH requires a valid position
(>=1).');
    END IF;
  ELSE
    IF :NEW.position IS NOT NULL THEN
      RAISE_APPLICATION_ERROR(-20111, 'DNF/DSQ must have NULL
position.');
    END IF;
    :NEW.points := 0;
  END IF;
END;
/
begin
dbms_output.put_line('Pirvan Sebastian');
end;
/
```

Creare-tabele.SQL | Triggers.sql M | Inserare-Date.SQL | testare.sql | UPDATE f1_race_results Untitle

```sql
UPDATE f1_race_results
 SET status='DNF', position=3
 WHERE race_id = (SELECT race_id FROM f1_races WHERE gp_code='MONACO_2026')
    AND driver_id = (SELECT driver_id FROM f1_drivers WHERE driver_code='VER');
-- USER is "PIRVANM_57";
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | QUERY RESULT | SCRIPT OUTPUT | SQL HISTORY | TASK MONITOR

```
Error starting at line : 1 in command -
UPDATE f1_race_results
   SET status='DNF', position=3
 WHERE race_id = (SELECT race_id FROM f1_races WHERE gp_code='MONACO_2026')
   AND driver_id = (SELECT driver_id FROM f1_drivers WHERE driver_code='VER')
Error at Command Line : 1 Column : 8
Error report -
SQL Error: ORA-20111: DNF/DSQ must have NULL position.
ORA-06512: at "PIRVANM_57.TRG_F1_VALIDATE_STATUS_POSITION", line 8
ORA-04088: error during execution of trigger 'PIRVANM_57.TRG_F1_VALIDATE_STATUS_POSITION'
```

Aplicație Oracle APEX (minim 4 pagini cu realizări diferite)

-

Formular Introducere Echipe (FORM):

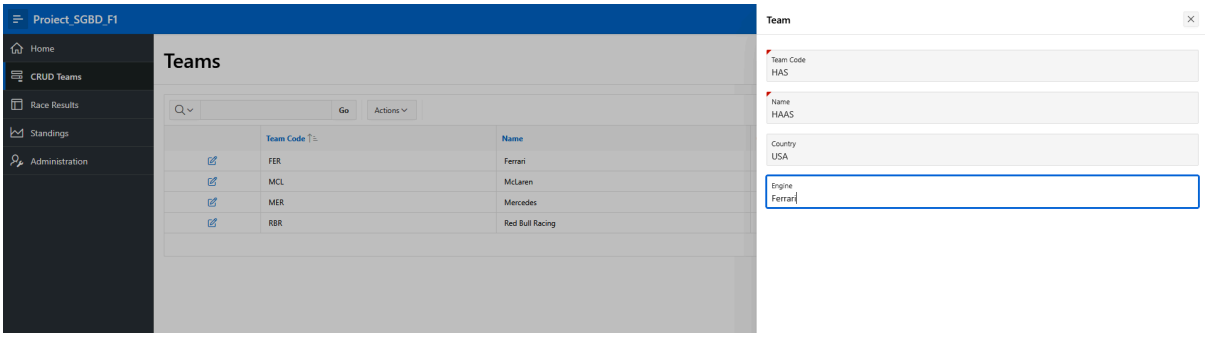
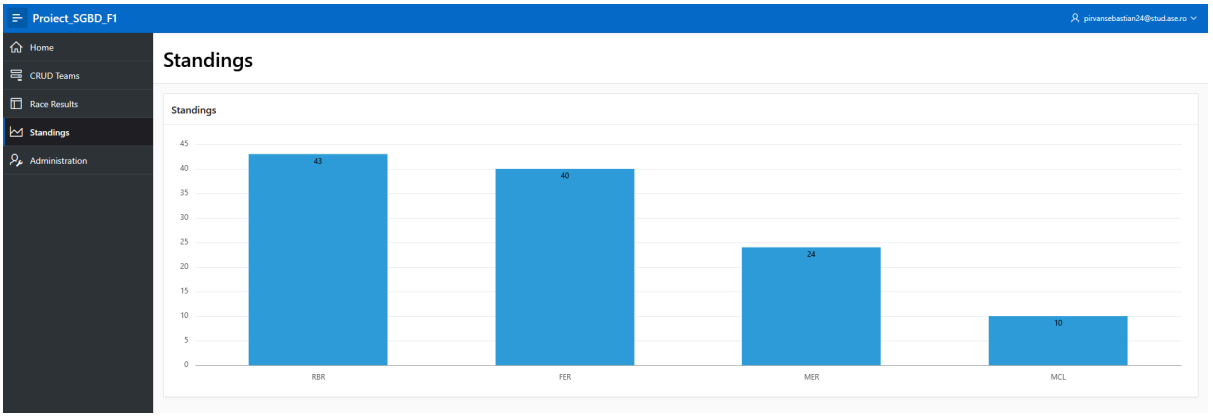
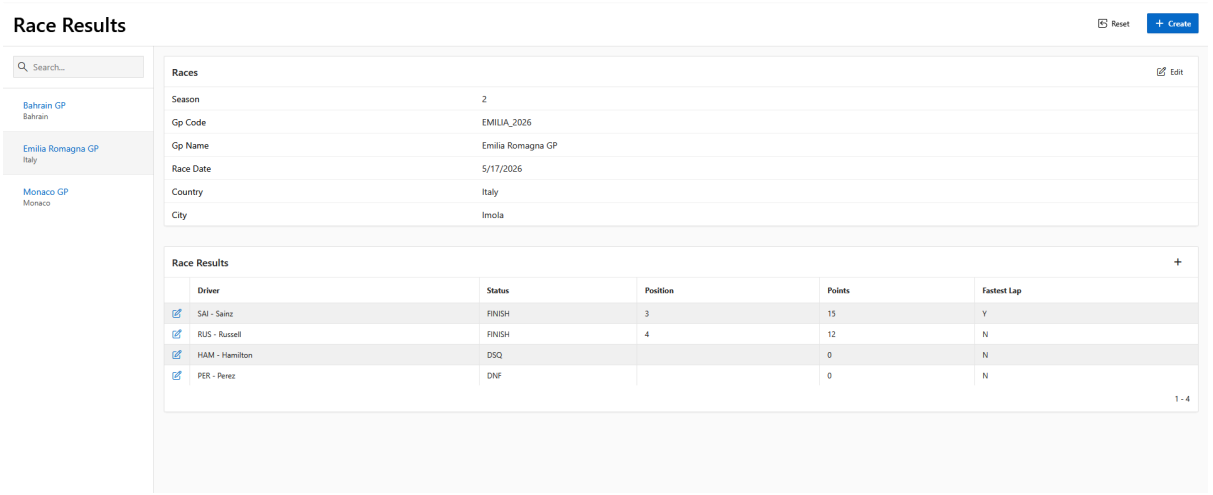
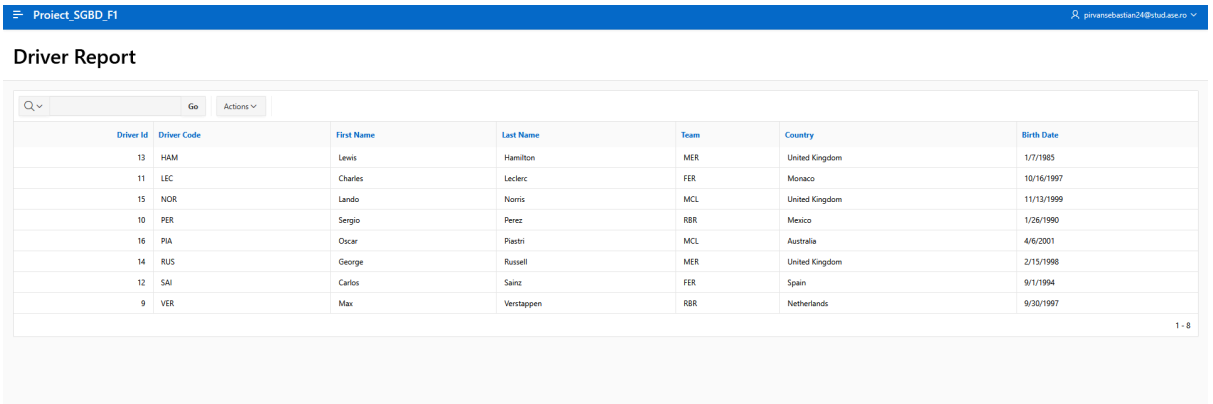


Chart Puncte campionat echipe (CHART):



Vizualizare Rezultate Cursa (Master-Detail):



Vizualizare Soferi(interactive-Report):